UNIVERSIDAD TECNOLÓGICA NACIONAL Facultad Regional Resistencia
Cátedra: Algoritmos y Estructuras de Datos | Ingeniería en Sistemas de Información

TRABAJO PRÁCTICO INTEGRADOR FINAL
Simulador de Misiones Artemis (Lenguaje C)

---

1. Fundamentación

El Trabajo Final Integrador tiene como propósito recuperar e integrar los contenidos desarrollados durante el cursado de la asignatura Algoritmos y Estructuras de Datos mediante la resolución de una problemática concreta y desafiante utilizando el lenguaje C.

La propuesta busca que las y los estudiantes puedan aplicar los conocimientos adquiridos en relación con estructuras de control, modularización avanzada, uso de tipos de datos simples y estructurados, validaciones y resolución algorítmica de problemas, promoviendo además el desarrollo de habilidades vinculadas al pensamiento crítico, la comunicación técnica, el trabajo colaborativo y la toma de decisiones dentro de un equipo de desarrollo de software.

Asimismo, la integración con la temática aeroespacial de la misión Artemis retoma los modelos de resolución de problemas físicos y simulación, alineando conceptos ya vistos en la asignatura Física I, exigiendo que el código refleje de manera rigurosa restricciones de dominio del mundo real. El trabajo pretende incorporar herramientas y dinámicas propias del desarrollo profesional de software, promoviendo el uso de repositorios Git/GitHub, la organización modular del código mediante archivos fuente (.c) y de cabecera (.h), y el uso responsable, ético y crítico de herramientas de Inteligencia Artificial como apoyo al aprendizaje y a la programación estructurada.

2. Objetivos de Aprendizaje

A través del desarrollo del Trabajo Final Integrador se espera que las y los estudiantes logren:

- **Aplicar de manera óptima** estructuras condicionales simples, compuestas y anidadas, así como estructuras repetitivas en la resolución de problemas de simulación algorítmica.
- **Diseñar e implementar** soluciones algorítmicas eficientes y robustas en lenguaje C respetando las buenas prácticas de programación estructurada y legibilidad de código.
- **Modularizar de forma avanzada** programas mediante funciones con responsabilidades bien definidas, aplicando paso de parámetros.
- **Diseñar y manipular** estructuras de datos para representar las entidades y estados de los sistemas simulados.
- **Implementar** mecanismos de validación de entradas de usuario y control estricto de requisitos del problema, para evitar comportamientos indefinidos o cierres inesperados del sistema.

- **Fortalecer el pensamiento crítico** una postura crítica frente a las soluciones sugeridas por herramientas de Inteligencia Artificial Generativa, pudiendo adaptar, optimizar y refactorizar el código propuesto.
- **Trabajar colaborativamente** en un entorno colegiado de desarrollo dividiendo tareas de forma equitativa y coordinada.
- **Comunicar** el diseño algorítmico, las estructuras de datos seleccionadas y las decisiones de implementación tomadas frente al trabajo final integrador.
- **Utilizar** plataformas de control de versiones (GitHub o GitLab) como herramienta básica de organización, control de cambios e historial progresivo de trabajo.
- **Comprender integralmente** la arquitectura interna, el flujo de ejecución y la gestión de memoria del sistema desarrollado.

3. Modalidad de Trabajo

- **Conformación de equipos:** El trabajo deberá realizarse obligatoriamente en grupos de entre 3 y 5 integrantes, ni más ni menos. Los equipos deberán conformarse vía aula virtual, y como lo deseen sus integrantes.
- **Escenario asignado:** Cada equipo desarrollará un sistema interactivo en forma de juego o simulador por consola basado en una de las 4 situaciones problemáticas de la misión ARTEMIS contempladas por la cátedra. Los escenarios serán asignados aleatoriamente por el equipo docente.
- **Entorno y Lenguaje:** Todos los sistemas deberán desarrollarse utilizando el lenguaje C y ejecutarse nativamente por consola o terminal de comandos.
- **Requisitos Técnicos Obligatorios:** 1) Estructuras condicionales y repetitivas; 2) Uso de modularización; 3) Uso de arreglos y punteros; 4) Validaciones exhaustivas de entrada; 5) Manejo de errores e interfaz limpia de mensajes al usuario.
- **Experiencia de Usuario:** Se espera que cada grupo diseñe una solución funcional, interactiva y coherente con el escenario asignado de Artemis, incorporando un menú principal, ciclo de juego/simulación por turnos o eventos, y generación de reportes finales de desempeño.

4. Escenarios Propuestos (Situaciones Problemáticas ARTEMIS)

Cada grupo deberá desarrollar el proyecto basado en una de las siguientes 4 situaciones problemáticas aeroespaciales:

Situación - Problema 1: Ascenso y Propulsión con el SLS (MRUV y Desprendimiento)

Durante el despegue desde el Centro Espacial Kennedy, el cohete Space Launch System (SLS) eleva a la nave Orion partiendo del reposo (v_0 = 0,0 m/s) en la plataforma de lanzamiento (y_0 = 0,0 m). En la primera fase de ascenso vertical rectilíneo, el cohete acelera hacia arriba con una aceleración constante de a_1 = 20,0 m/s² impulsado por los motores principales y los

dos cohetes de combustible sólido (SRB). A los t_1 = 120,0 s (2,00 min) de vuelo, los cohetes SRB agotados se desacoplan limpiamente. A partir de ese instante, el motor del núcleo principal continúa acelerando la nave a a_2 = 12,5 m/s² hasta alcanzar una altitud de h = 160,0 km (160.000,0 m), donde los motores se apagan. Se desprecia la resistencia del aire en la alta atmósfera.

Ejemplos de preguntas de análisis y calculo analítico que el programa deberia responder, a partir de distintos valores de entrada ingresados por el usuario

a. ¿Qué rapidez v_1 alcanza la nave al momento del desacople de los cohetes SRB (t = 120,0 s) y qué altitud y_1 ha recorrido hasta ese punto?
b. ¿Cuál es la rapidez máxima v_2 que alcanza la nave Orion al apagar sus motores a los h = 160.000,0 m de altitud?
c. Si los motores fallaran repentinamente al alcanzar los 160.000,0 m de altura y la nave quedara en caída libre bajo la aceleración de la gravedad (g = 9,80 m/s²), ¿cuál sería la altitud máxima hmáx alcanzada respecto de la plataforma de lanzamiento antes de comenzar a caer?
d. Exprese la aceleración máxima experimentada por la tripulación durante la primera fase como múltiplo de la gravedad terrestre (g).

Situación - Problema 2: Inserción y Estabilidad en Órbita Terrestre Baja — LEO (MCU y Gravitación)

Tras apagar sus motores de ascenso, la nave Orion es inyectada en una órbita terrestre baja (LEO) de estacionamiento, donde realiza un Movimiento Circular Uniforme (MCU) a una altitud constante de h = 192,0 km (192.000,0 m) sobre la superficie terrestre. Considere que la masa de la Tierra es M_T = 5,972 × 10^{24} kg, el radio terrestre medio es R_T = 6.371,0 km (6.371.000,0 m), la radio orbital total es r = R_T + h = 6.563,0 km (6.563.000,0 m), y la constante de gravitación universal es G = 6,674 × 10^{-11} N·m²/kg². La masa total de la nave Orion en órbita es m_{orion} = 33.000,0 kg.

Ejemplos de preguntas de análisis y cálculo analítico que el programa debería responder, a partir de distintos valores de entrada ingresados por el usuario

a. Demuestre analíticamente que la fuerza de atracción gravitatoria actúa como la fuerza centrípeta del sistema y calcule el módulo de la velocidad orbital v necesaria para mantener dicha órbita circular.
b. ¿Cuántos segundos y minutos le toma a la nave Orion completar una vuelta completa (período orbital T) alrededor de la Tierra?
c. Calcule el módulo de la aceleración centrípeta que experimentan los astronautas dentro de la cápsula a esa altitud.

d. Si durante la órbita la nave enciende un propulsor auxiliar lateral que genera un pulso de aceleración a = 0,50 m/s² constante y perpendicular a su velocidad orbital durante Δt = 10,0 s, describa la trayectoria resultante y calcule la desviación lateral instantánea.

Situación - Problema 3: Inyección Translunar — TLI e Inercia en Trayectoria Libre (1.ª Ley de Newton y Energía)

Estando en órbita baja (LEO), la etapa superior ICPS ejecuta la maniobra de Inyección Translunar (TLI), impulsando a la nave Orion hasta una velocidad de escape/inyección v_TLI = 10,9 km/s (10.900,0 m/s). Inmediatamente después del encendido, la etapa ICPS se desacopla y los motores principales se apagan por completo. La nave Orion (m = 26.000,0 kg) entra en una trayectoria de retorno libre (Free Return Trajectory) hacia la Luna, ubicada a una distancia media Tierra-Luna de D = 384.000,0 km (384.000.000,0 m).

Ejemplos de preguntas de análisis y cálculo analítico que el programa debería responder, a partir de distintos valores de entrada ingresados por el usuario

a. De acuerdo con la 1.ª Ley de Newton (Ley de Inercia), explique el comportamiento del estado de movimiento de la nave Orion una vez apagados los propulsores y describa cómo la fuerza gravitatoria terrestre actúa desacelerándose progresivamente.
b. Calcule la energía cinética inicial k_i que posee Orion al iniciar su viaje translunar a v = 10.900,0 m/s.
c. Mediante el principio de conservación de la energía mecánica (Em = K + U), determine el trabajo adicional mínimo Wext por unidad de masa requerido para desplazar la nave desde la órbita baja (h = 192.000,0 m) hasta el punto de equilibrio gravitatorio Tierra-Luna.
d. Justifique desde la perspectiva energética por qué se requiere una masa significativamente mayor de combustible para viajar de la Tierra a la Luna que para el viaje de regreso Luna-Tierra.

Situación - Problema 4: Reingreso Atmosférico, Frenado Aerodinámico y Amerizaje (Trabajo, Energía e Impacto)

Tras completar el sobrevuelo lunar, el módulo de tripulación Orion (m = 10.400,0 kg) se separa del módulo de servicio y reingresa a la atmósfera terrestre a una velocidad hiperveloz de vi = 11,0 km/s (11.000,0 m/s, equivalente a 39.600,0 km/h o Mach 32) a una altitud de h = 120,0 km (120.000,0 m). El escudo térmico disipa la energía cinética mediante el trabajo de la fuerza de resistencia aerodinámica del aire (Froz), reduciendo la rapidez hasta vf = 25,0 m/s al momento de desplegar los paracaídas principales a una altitud de h = 2,50 km (2.500,0 m). Finalmente, la nave impacta en la superficie del Océano Pacífico (amerizaje) a una rapidez final segura de vimpacto = 8,0 m/s.

Ejemplos de preguntas de análisis y cálculo analítico que el programa debería responder, a partir de distintos valores de entrada ingresados por el usuario

a. Calcule el trabajo total Wroz realizado por la fuerza de rozamiento atmosférico sobre la cápsula desde el reingreso (120.000,0 m) hasta la apertura de los paracaídas (2.500,0 m).
b. Si la fase de mayor desaceleración dura $\Delta t = 400{,}0$ s, determine la desaceleración media ejercida sobre la tripulación en m/s² y en múltiplos de la gravedad (g).
c. Suponiendo que durante el amerizaje la cápsula se hunde d = 1,50 m en el agua hasta detenerse momentáneamente antes de flotar, determine la fuerza promedio Fprom que el agua ejerce sobre la estructura. Exprese el resultado en Newtons (N) y como múltiplo del peso de la nave.
d. Analice qué sucedería si el ángulo de reingreso γ estuviera fuera de la ventana de seguridad ($5{,}5° \leq \gamma \leq 7{,}0°$): evalúe las consecuencias físicas si $\gamma > 7{,}0°$ (desaceleración excesiva) o si $\gamma < 5{,}5°$ (rebote atmosférico)

5. Requisitos Técnicos y Arquitectura en C

Para garantizar los estándares académicos de la materia Algoritmos y Estructuras de Datos, la solución deberá cumplir con los siguientes lineamientos técnicos de programación en C:

5.1. Modularización y Archivos del Proyecto

El proyecto no podrá desarrollarse en un único archivo mono-bloque. Debe fragmentarse en módulos con separación lógica de responsabilidades:

- **Módulo Principal (main.c):** Contiene únicamente la función main(), el menú principal de navegación y la inicialización del flujo general del programa.
- **Módulo de Cabecera (simulador.h / artemis.h):** Declaración de constantes (#define), definición de tipos de datos estructurados (struct, typedef), y prototipos de funciones públicas de la simulación.
- **Módulo de Lógica (simulador.c / artemis.c):** Implementación de los algoritmos de simulación, actualización del estado del juego, cálculos físicos y lógica de negocio.
- **Módulo de Validaciones e Interfaz (validaciones.c / .h):** Funciones dedicadas al ingreso seguro de enteros, flotantes y caracteres, y mensajes de advertencia.

5.2. Control de Versiones con Git/GitHub

Cada grupo deberá trabajar obligatoriamente en un repositorio público de GitHub o GitLab:

- **Historial Activo:** El repositorio debe evidenciar un desarrollo gradual con múltiples commits realizados por todos los miembros del equipo a lo largo del período de trabajo. No se aceptarán repositorios subidos en un único commit final.

- **Trabajo Colaborativo:** Uso recomendado de ramas (feature branches) y Pull Requests para integrar componentes del software.

6. Uso de Inteligencia Artificial

La utilización de herramientas de Inteligencia Artificial Generativa (ChatGPT, Claude, GitHub Copilot, Gemini, etc.) se encuentra permitida como apoyo al aprendizaje y al desarrollo del proyecto.

Las herramientas de IA podrán utilizarse para:

- Generar ideas iniciales sobre la interfaz de consola o mecánicas del juego.
- Resolver dudas conceptuales sobre la sintaxis de C, punteros o manejo de memoria.
- Analizar mensajes de error del compilador GCC o segfaults.
- Asistir en la depuración y confección de casos de prueba.

> **IMPORTANTE - TRANSPARENCIA EN EL USO DE IA**
> *Sin embargo, todos los integrantes del grupo deberán comprender integralmente la solución implementada y poder defender técnicamente cada línea de código durante el coloquio, en caso de solicitarse. La IA no reemplaza la necesidad de dominar la lógica algorítmica y el código generado para el programa presentado.*

Deberán registrar obligatoriamente en la sección correspondiente del archivo README.md qué herramientas de IA han utilizado, para qué tareas específicas, qué prompts o consultas realizaron y cómo validaron/adaptaron el código generado.

7. Entregables

La entrega digital se realizará formalmente a través de la plataforma virtual habilitada por la cátedra en la fecha indicada.

Cada equipo deberá presentar:

1. Enlace al Repositorio GitHub / GitLab: Debe contener todo el código fuente modularizado (.c y .h), archivo Makefile o scripts de compilación, y el historial de cambios.
2. Archivo README.md: Documento Markdown ubicado en la raíz del repositorio conteniendo: a) Integrantes y comisión; b) Descripción detallada del escenario Artemis asignado y reglas del juego; c) Breve detalle de funcionamiento; d) Instrucciones claras de compilación y ejecución; e) Registro detallado del uso de Inteligencia Artificial.
3. Video grabado por el equipo completo, con una duración máxima de 5 minutos, subido a YouTube, Drive o Loom. El video debe incluir:

a) Presentación breve de cada integrante, con aparición en el video y utilizando su voz real.

b) Explicación general de la arquitectura del programa en C y su modularización.
c) Demostración de la ejecución del juego mediante un caso exitoso.
d) Demostración de las validaciones implementadas ante entradas erróneas del usuario.

Condiciones de realización: La presentación debe realizarse mediante un video que registre la participación real de los integrantes del equipo. No está permitido utilizar inteligencia artificial para generar, clonar o modificar voces con el fin de reemplazar las voces reales de los integrantes, ni para generar avatares, personajes o simulaciones de personas que aparenten ser miembros del equipo. Las presentaciones y explicaciones deben ser realizadas por los propios estudiantes. Se permite el uso de herramientas de edición de video, siempre que no se utilicen para falsear la participación de los integrantes ni para sustituirla por contenido generado mediante inteligencia artificial.

8. Modalidad de Evaluación y Coloquio

La evaluación del Trabajo Final Integrador se realizará en dos instancias complementarias: Entrega Digital y Coloquio de Defensa.

Posteriormente a la revisión digital, la cátedra convocará a algunos equipos a un coloquio presencial o sincrónico, de acuerdo a criterios definidos por la cátedra. Durante el coloquio, uno o más integrantes del grupo serán seleccionados individualmente por el tribunal docente para defender el proyecto.

9. Rúbrica de Evaluación

La calificación del Trabajo Final Integrador se determinará con base en los siguientes criterios disciplinares:

Criterio de Evaluación	Sobresaliente (10-9)	Logrado (8-7)	En Desarrollo (6)	No Logrado (1-5)
Diseño Algorítmico y Modularización (C)	Excelente división en .h y .c. Funciones óptimas, sin código duplicado. Uso impecable de structs y punteros.	Buena modularización. Uso correcto de structs y punteros con mínimos detalles a mejorar.	Modularización deficiente o todo en 1 o 2 archivos. Punteros mal aplicados o escasos.	Código monobloque. Sin funciones, sin structs ni punteros. No compila o presenta errores graves.
Validaciones e Interfaz de Consola	Validación total contra entradas erróneas (tipo y rango). Interfaz de consola clara, intuitiva y fluida.	Valida la mayoría de datos. La consola es comprensible y funcional.	Validaciones básicas o incompletas. Errores al ingresar letras por números.	Sin validaciones. El programa se rompe o entra en bucle infinito ante errores de usuario.

Fidelidad al Escenario Artemis	Simulación física/operativa fiel a la problemática Artemis elegida. Lógica rica y juego desafiante.	Representa adecuadamente el escenario Artemis asignado con mecánicas claras.	Simulación muy simplificada o que se aleja de la problemática planteada.	No responde a ninguna situación problemática de Artemis o es irrelevante.
Uso de Git, GitHub y README	Historial con múltiples commits equitativos. README impecable con diagrama, instrucciones y registro de IA.	Repositorio ordenado, commits frecuentes de los integrantes. README completo.	Pocos commits o concentrados en un solo integrante. README incompleto.	Un solo commit final. Sin README o repositorio privado sin acceso.
Defensa Técnica en Coloquio	Dominio absoluto del código C, justificación técnica sólida de cada decisión y uso crítico de IA.	Comprende y explica adecuadamente el código implementado y las estructuras C utilizadas.	Comprensión parcial del código. Dificultad para explicar el uso de punteros o módulos.	No logra explicar el código presentado. Desconocimiento total de la implementación.